

pypomp: Inference for partially observed Markov process models in Python with JAX

DRAFT IN PROGRESS

Aaron J. Abkemeier* Jun Chen* Kevin Tan Jesse Wheeler
Bo Yang Kunyang He Jonathan Terhorst Aaron A. King
Edward L. Ionides *These authors contributed equally

Methods for fitting and evaluating partially observed Markov process (POMP) mechanistic models are essential across fields like epidemiology, ecology, and finance, but are often hindered by high computational demands. We introduce **pypomp**, a high-performance Python package for statistical inference using POMP models. Designed for complex stochastic dynamic systems, **pypomp** implements plug-and-play, likelihood-based algorithms including sequential Monte Carlo and iterated filtering. By leveraging GPU acceleration, **pypomp** significantly outperforms its predecessor, the R package **pomp**. Models that previously required months to fit now complete in days, enabling the use of formerly impractical models. Furthermore, **pypomp** utilizes JAX to perform gradient descent on differentiable particle filters, achieving likelihood maximization beyond the practical limits of iterated filtering alone. The package provides a comprehensive interface that streamlines model construction, fitting, and analysis, allowing practitioners to develop and evaluate complex models with minimal implementation overhead.

1 Introduction

Partially Observable Markov Process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling time-series dynamic systems where the underlying state process is not directly observed. Methods such as particle filtering (Arulampalam et al. 2002) and iterated filtering (E. L. Ionides, Breto, and King 2006; Edward L. Ionides et al. 2015) have been developed to perform maximum likelihood estimation and evaluation for POMP models. A defining strength of these methods is that they are performed by simulating from the process model instead of evaluating exact transition probabilities (a property known as plug-and-play (Bretó et al. 2009)), which makes

them suitable for examining a wide range of models unconstrained by needs for mathematical convenience. Consequently, POMP models find extensive application in epidemiology (Mitchen et al. 2024; Fox et al. 2022; Wen et al. 2024), ecology (Auger-Méthé et al. 2021; Marino et al. 2019; Blackwood et al. 2013), finance (Bretó 2014), and other domains where mechanistic modeling is desired.

Thus far, the `pomp` package ecosystem in R has provided a basis for POMP modeling of several types. The `pomp` package is designed for standard POMP models, while its extension packages `panelPomp`, `spatpomp`, and `phyloPomp` further enhance its capabilities for panel, spatio-temporal, and phylodynamic data analysis, respectively (King, Nguyen, and Ionides 2016; Bretó et al. 2025; Asfaw et al. 2024).

While powerful, statistical inference for POMP models poses substantial computational challenges. To achieve a parameter estimate which converges to the maximum likelihood estimate (MLE), it is often necessary to run Iterated Filtering 2 (IF2) for hundreds of iterations, utilizing thousands of particles for each iteration, starting from dozens of initial parameter estimates. One full pass through a time series may involve hundreds of simulation steps per particle. The aforementioned R packages are built only to use central processing unit (CPU) devices, which are relatively slow for these methods even when running independent method calls in parallel. Consequently, these methods are bottlenecked by their runtime. It may then be necessary to compromise on the number of particles or iterations used, or to accept that some jobs – such as those that run on the scale of months – are simply infeasible. Even if a job can finish in an acceptable time, the latency from submitting a job to receiving the results can slow down the pace of iterative model development, and the monetary cost of running lengthy jobs on advanced hardware can add up.

As it turns out, POMP methods can be sped up considerably. Many of the processes involved in POMP methods are embarrassingly parallel, such as simulating the state process for each of thousands of particles, running the particle filter multiple times for the same parameter set, and running iterated filtering from multiple starting parameter sets. This last point is holds true especially when estimating a profile likelihood to construct a confidence interval as in E. L. Ionides et al. (2017). Graphics processing unit (GPU) devices are well-suited for embarrassingly parallel operations of this scale.

To address the need for plug-and-play POMP methods that leverage GPU acceleration, we introduce the package `pypomp` (Abkemeier et al. 2024). Briefly stated, it is a Python implementation of the core algorithms in `pomp` and `panelPomp`, but written such that they can be run on both GPUs and CPUs. We accomplish this by using JAX (Bradbury et al. 2018), a high-performance numerical computing library that supports hardware acceleration and automatic vectorization. [AA: Need to change "90 times faster" to something more specific after we more rigorously compare `pypomp` with `pomp`. In addition, I think we should include the @korevaar2020 data set fit in this paper, as it provides perhaps the strongest motivation for developing `pypomp`.] The resulting package runs methods up to 90 times faster on GPUs than `pomp` can on CPUs. Jobs that would take months in `pomp` can be completed in days in `pypomp`, enabling plug-and-play likelihood-based inference on data sets that would otherwise be intractably large. This

is demonstrated in Section [AA: add this section], where we fit a panel POMP model to the Korevaar, Metcalf, and Grenfell (2020) England and Wales measles dataset, which contains over one million observations. While this would normally take over 5 months in `pomp`, `pypomp` can finish the job in 41 hours [AA: revise these numbers as appropriate].

The remainder of this paper is organized as follows. Section 2 demonstrates the mathematical notation for POMP models and their related implementation in `pypomp`. Section 3 introduces the embedded methodologies. Section 4 presents data analysis workflows and benchmarking results. ?@sec-discussion concludes with a discussion of future work.

2 POMP Models in pypomp

A partially observed Markov process (POMP) model is defined by an initial density $f_{X_0}(x_0; \theta)$, a transition density $f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta)$, and a measurement density $f_{Y_n|X_n}(y_n | x_n; \theta)$. In `pypomp`, these are implemented as an object of class `Pomp`, which encapsulates the model mechanics and parameters.

While users can define custom models by providing JAX-compatible Python functions for each component, `pypomp` we demonstrate the API with the premade S&P500 model.

```
import pypomp as pp
import jax

spx_obj = pp.models.spx()

key = jax.random.key(1)
key, subkey = jax.random.split(key)
```

2.0.1 Panel POMP class

3 POMP Inference Methods

`pypomp` implements several core likelihood-based inference methods, all of which leverage the **plug-and-play property** (E. L. Ionides, Breto, and King 2006) and JAX-powered acceleration. These methods are implemented as methods of the `Pomp` class and modify the object in place by appending results to `results_history`.

1. **Particle Filter (`pfilter`)**: A standard sequential Monte Carlo (SMC) algorithm for likelihood evaluation and state estimation. It provides unbiased Monte Carlo estimates of the log-likelihood.

2. **Iterated Filtering (mif)**: An implementation of the IF2 algorithm (Edward L. Ionides et al. 2015) for maximum likelihood estimation. It combines particle filtering with sequential random-walk perturbations and a cooling schedule to explore the parameter space.
3. **Automatic Differentiation (mop and train)**: `pypomp` introduces a differentiable variant of the particle filter (MOP- α , Tan, Hooker, and Ionides (2024)) that enables the use of JAX’s automatic differentiation. The `train()` method utilizes these gradients (and optionally Hessians) for local optimization.

By utilizing JAX, these methods achieve high performance via JIT compilation and vectorization, allowing them to run efficiently on both CPUs and GPUs. The following example demonstrates a typical inference workflow: starting with global exploration via IF2, followed by gradient-based local refinement, and concluding with a final likelihood evaluation.

```
rw_sd = pp.RWSigma(
    sigmas={
        "mu": 0.0001,
        "kappa": 0.01,
        "theta": 0.00001,
        "xi": 0.001,
        "rho": 0.05,
        "V_0": 0.001,
    },
    init_names=["V_0"],
)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, a=0.5, key=subkey)

eta = {name: 0.001 for name in spx_obj.canonical_param_names}
spx_obj.train(J=1000, M=10, eta=eta, optimizer="Adam")

spx_obj.pfilter(J=1000, reps=10, key=subkey)

print(spx_obj.traces().tail())
```

	theta_idx	iteration	method	logLik	mu	kappa	theta	xi	rho	V_0
17	0	17	train	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	0	18	train	NaN	NaN	NaN	NaN	NaN	NaN	NaN
19	0	19	train	NaN	NaN	NaN	NaN	NaN	NaN	NaN
20	0	20	train	NaN	NaN	NaN	NaN	NaN	NaN	NaN
21	0	20	pfilter	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Each method call generates a result object (e.g., `PompPFilterResult`, `PompMIFResult`, `PompTrainResult`) which contains execution time, parameter traces, and log-likelihood

estimates. These objects are derived from a common base class and provide helper functions like `results()`, `traces()`, and `print_summary()`. This unified architecture ensures that complex multi-stage inference workflows are easily manageable.

4 Data Analysis with pypomp / Tutorials and data analysis examples

We first show the benchmarking results of our software running on the *dacca* dataset.

Table: Main arguments to the `Pomp` class and related constructor objects.

For *mif2*, *train* and *ifad*, we started with a param box including the true (default) parameters, 0.5 x true parameters and 1.5 x true parameters. Each replicate, we chose the best loglikelihood estimate among the three starting points. In this table, the *pfilter* and *mop* are evaluated at the true ()

Software	Backend	Run Level	Method	Evaluated Loglikli-hood (logexp-mean)	SE	Average Run Time(s)	Maximum Memory Usage (MiB)
pypomp	CPU	3	pfilter	-3748.41	0.43	2.23	739
pypomp	CPU	3	mop	-3748.02	0.32	3.31	713
pypomp	CPU	3	IF2	-3770.37	1.00	639.52	834
pypomp	CPU	3	pure train()				
pypomp	CPU	3	IFAD				

```

pypomp | GPU | 3 | pfilter | -3747.58 | 0.35 | 0.26 | 947 |
pypomp | GPU | 3 | mop | -3747.61 | 0.71 | 0.39 | 745 |
pypomp | GPU | 3 | IF2 | -3763.69 | 0.42 | 28.90 | 1638.4 |
pypomp | GPU | 3 | pure train() | -20000 | | | |
pypomp | GPU | 3 | IFAD | | | | |

pypomp | CPU | 4 | pfilter | -3748.16 | 0.13 | 7.32 | 852 |
pypomp | CPU | 4 | mop | -3742.11 | 0.15 | 14.53 | 790 |
pypomp | CPU | 4 | mif2 | -3752.48 | 0.27 | 198.41 | 1126.4 |

pypomp | GPU | 4 | pfilter | -3748.15 | 0.18 | 0.41 | 857 |

pypomp | GPU | 4 | mop | -3748.34 | 0.12 | 0.49 | 821 |
pypomp | GPU | 4 | mif2 | -3752.48 | 0.27 | 198.41 | 2048 |

```

Here the optimizer for `train()` and `ifad()` is set to the Adam optimizer. The maximum memory usage is obtained from Greatlakes summary. From the table, we can see three points: 1. **Statistical Correctness:** The log-likelihood estimates from `pfilter()` and `mop()` are close to each other, with `mop()` showing slightly higher values, which is expected as `mop()` is designed to provide a differentiable likelihood estimate that can be optimized. 2. **IF2 Performance:** The IF2 method shows a lower log-likelihood estimate compared to `pfilter()` and `mop()`, which may be due to the fact that IF2 is an iterative method that may not have fully converged within the given number of iterations, or it may be more sensitive to the choice of starting parameters and cooling schedule. But overall, the IF2 captures the overall likelihood structure and provides a reasonable estimate, albeit with higher variance. 3. **GPU Acceleration:** The GPU implementations of `pfilter()` and `mop()` show significantly reduced run (give the CPU/GPU run time ratio). 4. **Method Comparison:** (Dmop, IFAD, IF2...)

We also use the R package `pomp` to run the same methods as the baseline and compare the results with `pypomp`.

This section demonstrates:

- Log-likelihood profiling
- GPU benchmarking
- Conditional log-likelihood residuals # Discussion {#sec-discussion}

- Abkemeier, Aaron, Jun Chen, Edward Ionides, Jesse Wheeler, and Kevin Tan. 2024. “Py-pomp.”
- Arulampalam, M. S., S. Maskell, N. Gordon, and T. Clapp. 2002. “A Tutorial on Particle Filters for Online Nonlinear/Non-Gaussian Bayesian Tracking.” *IEEE Transactions on Signal Processing* 50 (2): 174–88. <https://doi.org/10.1109/78.978374>.
- Asfaw, Kidus, Joonha Park, Aaron A. King, and Edward L. Ionides. 2024. “spatPomp: An R Package for Spatiotemporal Partially Observed Markov Process Models.” *Journal of Open Source Software* 9 (104): 7008. <https://doi.org/10.21105/joss.07008>.
- Auger-Méthé, Marie, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, et al. 2021. “A Guide to State-Space Modeling of Ecological Time Series.” *Ecological Monographs* 91 (4): e01470. <https://doi.org/10.1002/ecm.1470>.
- Blackwood, J. C., D. G. Streicker, S. Altizer, and P. Rohani. 2013. “Resolving the Roles of Immunity, Pathogenesis, and Immigration for Rabies Persistence in Vampire Bats.” *Proceedings of the National Academy of Sciences of the United States of America* 110 (51): 20837–42. <https://doi.org/10.1073/pnas.1308817110>.
- Bradbury, James, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, et al. 2018. “JAX: Composable Transformations of Python+NumPy Programs.”
- Bretó, Carles. 2014. “On Idiosyncratic Stochasticity of Financial Leverage Effects.” *Statistics & Probability Letters* 91: 20–26. <https://doi.org/10.1016/j.spl.2014.04.003>.

- Bretó, Carles, Daihai He, Edward L. Ionides, and Aaron A. King. 2009. “Time Series Analysis via Mechanistic Models.” *The Annals of Applied Statistics* 3 (1): 319–48. <https://doi.org/10.1214/08-AOAS201>.
- Bretó, Carles, Jesse Wheeler, Aaron A. King, and Edward L. Ionides. 2025. “panelPomp: Analysis of Panel Data via Partially Observed Markov Processes in R.” arXiv. <https://doi.org/10.48550/arXiv.2410.07934>.
- Fox, S. J., M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, et al. 2022. “Real-Time Pandemic Surveillance Using Hospital Admissions and Mobility Data.” *Proceedings of the National Academy of Sciences* 119 (7): e2111870119. <https://doi.org/10.1073/pnas.2111870119>.
- Ionides, E. L., C. Breto, and A. A. King. 2006. “Inference for Nonlinear Dynamical Systems.” *Proceedings of the National Academy of Sciences* 103 (49): 18438–43. <https://doi.org/10.1073/pnas.0603181103>.
- Ionides, E. L., C. Breto, J. Park, R. A. Smith, and A. A. King. 2017. “Monte Carlo Profile Confidence Intervals for Dynamic Systems.” *Journal of The Royal Society Interface* 14 (132): 20170126. <https://doi.org/10.1098/rsif.2017.0126>.
- Ionides, Edward L., Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. 2015. “Inference for Dynamic and Latent Variable Models via Iterated, Perturbed Bayes Maps.” *Proceedings of the National Academy of Sciences* 112 (3): 719–24. <https://doi.org/10.1073/pnas.1410597112>.
- King, Aaron A., Dao Nguyen, and Edward L. Ionides. 2016. “Statistical Inference for Partially Observed Markov Processes via the R Package **Pomp**.” *Journal of Statistical Software* 69 (12). <https://doi.org/10.18637/jss.v069.i12>.
- Korevaar, Hannah, C. Jessica Metcalf, and Bryan T. Grenfell. 2020. “Structure, Space and Size: Competing Drivers of Variation in Urban and Rural Measles Transmission.” *Journal of The Royal Society Interface* 17 (168): 20200010. <https://doi.org/10.1098/rsif.2020.0010>.
- Marino, J. A. Jr., S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. 2019. “Evaluating Consumptive and Non-consumptive Predator Effects on Prey Density Using Field Time-Series Data.” *Ecology* 100 (3): e02583. <https://doi.org/10.1002/ecy.2583>.
- Mietchen, Matthew S., Erin Clancey, Corrin McMichael, and Eric T. Lofgren. 2024. “Estimating SARS-CoV-2 Transmission Parameters Between Coinciding Outbreaks in a University Population and the Surrounding Community.” <https://doi.org/10.1101/2024.01.10.24301116>.
- Tan, Kevin, Giles Hooker, and Edward L. Ionides. 2024. “Accelerated Inference for Partially Observed Markov Processes Using Automatic Differentiation.” arXiv. <https://doi.org/10.48550/arXiv.2407.03085>.
- Wen, L., Y. Yin, Q. Li, Z. Peng, and D. He. 2024. “Modeling the Co-Circulation of Influenza and COVID-19 in Hong Kong, China.” *Advances in Continuous and Discrete Models* 2024 (1): 1–9. <https://doi.org/10.1186/s13662-024-03830-7>.